Лабораторная работа №2. Управление потоками

Вихарев Лев Александрович, М8О-214БВ-25. Вариант 3: ((3 − 1) % 20) + 1 = 3.

Цель и постановка задачи

Реализовать последовательную и параллельную сортировку слиянием массива целых чисел на языке C, ограничить количество потоков параметром запуска, продемонстрировать их средствами ОС и исследовать ускорение и эффективность при разных входных данных и числе потоков.

С учётом дополнительных указаний преподавателя в программе отсутствуют функции стандартной библиотеки C для ввода-вывода, выделения памяти, строк, преобразования чисел и генерации случайных данных. Использованы POSIX-интерфейсы write, mmap, munmap, clock_gettime, sysconf, _exit и POSIX Threads. Заголовки с определениями типов и констант не исключаются. POSIX-вызовы доступны через системную C-библиотеку; программа не является двоичным файлом без libc. Независимый тестовый стенд использует qsort и стандартную библиотеку вне сдаваемой программы.

Алгоритм и функции

merge_sort рекурсивно делит полуинтервал [left, right) пополам, сортирует обе половины и объединяет их функцией merge. Слияние сначала записывается в буфер, затем переносится в исходный массив. Время последовательной версии O(n log n), дополнительный буфер O(n), глубина рекурсии O(log n). Память стеков зависит также от числа потоков.

parallel_sort при длине участка не меньше 16384 и наличии бюджета создаёт один поток для левой половины, а правую сортирует текущий. После pthread_join выполняется слияние. Если участок меньше порога или бюджет исчерпан, вызывается merge_sort. Отдельные слияния не распараллеливаются.

threads_left изначально равен P−1. Проверка и уменьшение выполняются под мьютексом, бюджет не пополняется после завершения ребёнка. Таким образом, за весь запуск создаётся не больше P потоков с учётом главного. Это более строгое ограничение, чем максимум одновременно существующих потоков. Ожидающий родитель продолжает существовать и входит в предел. Фактическое число созданных потоков печатается отдельно от лимита.

Потоки используют непересекающиеся участки исходного массива и буфера. Мьютекс защищает только бюджет, pthread_join обеспечивает завершение ребёнка перед чтением его результата. ThreadArgs передаёт адреса общих массивов и границы; структура на стеке родителя действительна до join. Распределение бюджета зависит от порядка выполнения ветвей планировщиком, поэтому дерево может быть несимметричным.

main создаёт через mmap один блок для двух одинаковых копий массива и буфера, заполняет данные собственным 32-битным генератором с начальным состоянием 42, сортирует копии последовательно и параллельно, проверяет результаты и освобождает память через munmap. read_number разбирает десятичные цифры вручную с проверкой переполнения. equal сравнивает строки, write_text обрабатывает короткие записи и EINTR, print_int и print_value форматируют числа без printf. При ошибке pthread код ошибки печатается числом, процесс завершается через _exit.

Сборка, запуск и демонстрация

В Linux/WSL: make; ./merge 4 1000000. Формат: ./merge THREADS [SIZE] [random|sorted|reverse]. Диапазон аргумента потоков 1–4096, размер по умолчанию 5 000 000. Для маленькой демонстрации: ./merge 4 10 reverse. До 20 элементов печатаются исходный и отсортированный массивы; в остальных случаях — размер, лимит, число созданных потоков, логические процессоры, оба времени в мс, S, E и Check: OK/FAILED. При нехватке ресурсов pthread процесс сообщает об ошибке; измерение не подменяется успешным результатом.

python3 tools/check_threads.py наблюдает Threads в /proc/PID/status и выводит снимок ps -T для четырёх потоков. Для ручного наблюдения можно запустить ./merge 4 10000000 в фоне и повторять ps -T -p PID. Начальная последовательная часть и финальное слияние используют один поток. Сэмплирование может пропустить короткоживущие потоки; число созданий дополнительно учитывает сама программа.

make test: 45 сравнений функций с независимой qsort и 38 проверок интерфейса запуска, включая предел 1024 на коротком массиве, границы int, размеры вокруг порога и неправильные аргументы. Проверяется отсутствие запрещённых библиотечных вызовов в исполняемом файле через nm. Дополнительно проверен тестовый стенд с AddressSanitizer и UndefinedBehaviorSanitizer. Эти проверки не являются математическим доказательством отсутствия всех ошибок.

Методика эксперимента и метрики

Среда: 13th Gen Intel(R) Core(TM) i5-13420H; 12 логических процессоров; Linux-6.18.54.1-microsoft-standard-WSL2-x86_64-with-glibc2.39; gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0. Оптимизация -O2, -fno-builtin, сборка с -pthread. Число логических процессоров получено через nproc/sysconf и сохранено в environment.json.

Числа потоков: 1, 2, 4, 8 — четыре значения меньше 12; 12 — равно числу логических процессоров; 32, 64, 128 — три значения существенно больше 12. Для каждого размера 10 000, 1 000 000 и 5 000 000 исследованы случайные, возрастающие и убывающие данные. Всего 72 комбинации, по три запуска — 216 успешных запусков. Во всех случаях Check: OK.

Для каждого запуска сначала измеряется последовательная сортировка, затем параллельная на одинаковых данных. Часы CLOCK_MONOTONIC; в таблицах миллисекунды. Время включает сортировку, создание потоков, синхронизацию и копирование при слиянии. Генерация, выделение памяти, проверка, печать и освобождение в интервал не входят. Внешний наблюдатель читает /proc примерно раз в 1 мс; он добавляет фоновую нагрузку. Для каждой комбинации берутся отдельно медианы трёх последовательных и трёх параллельных времён.

Ускорение S(P) = Tseq / Tpar(P). Tseq — медиана времени последовательной версии для этой комбинации данных и P; Tpar(P) — медиана времени параллельной версии с лимитом P. Ускорение показывает, во сколько раз сократилось время: S > 1 — выигрыш, S < 1 — замедление. Последовательная база измеряется заново для каждой строки и хранится в CSV.

Эффективность E(P) = S(P) / P, безразмерная величина. Она нормирует ускорение на заданный бюджет потоков; E = 1 соответствует линейному ускорению, E = 0,25 — 25% от него. P — лимит, а не обязательно пик одновременно полезно работающих потоков. Если из-за порога потоки не созданы или ветви завершились раньше, E всё равно делится на

заданный P; это нужно учитывать при интерпретации.

Результаты измерений

10 000 элементов случайные

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	0.558003	0.980	0.980419
2	0.522168	1.022	0.510761
4	0.532610	1.082	0.270542
8	0.563344	1.017	0.127141
12	0.600940	0.926	0.077168
32	0.566726	0.949	0.029670
64	0.560956	0.969	0.015146
128	0.517591	1.038	0.008113

10 000 элементов по возрастанию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	0.131885	0.908	0.907988
2	0.109207	1.089	0.544626
4	0.113882	1.076	0.268996
8	0.114479	1.039	0.129887
12	0.108412	1.103	0.091896
32	0.114479	1.053	0.032906
64	0.122137	0.974	0.015218
128	0.107616	1.128	0.008809

10 000 элементов по убыванию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	0.130990	0.905	0.905084
2	0.126016	0.927	0.463298
4	0.115772	1.021	0.255155
8	0.106324	1.122	0.140200
12	0.117861	1.024	0.085372
32	0.127906	0.914	0.028577
64	0.114976	1.029	0.016071
128	0.107815	1.097	0.008569

1 000 000 элементов случайные

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	84.869151	0.946	0.945775
2	44.028944	1.737	0.868412
4	40.079533	2.097	0.524149
8	25.375595	3.103	0.387850
12	24.200606	3.293	0.274434
32	21.876072	3.881	0.121280
64	18.056677	4.271	0.066733
128	20.199784	3.854	0.030107

1 000 000 элементов по возрастанию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	15.098318	1.077	1.076902
2	9.158288	1.922	0.961226
4	9.199195	2.154	0.538433
8	5.982236	2.731	0.341318
12	6.857366	2.544	0.212004
32	8.071034	2.024	0.063258
64	7.603287	2.180	0.034057
128	7.481661	2.309	0.018042

1 000 000 элементов по убыванию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	16.461456	1.129	1.128527
2	9.274866	1.746	0.873007
4	8.322945	2.273	0.568143
8	6.711679	2.417	0.302100
12	6.563692	2.610	0.217470
32	6.393739	2.785	0.087040
64	7.934907	2.234	0.034905
128	8.407522	1.983	0.015492

5 000 000 элементов случайные

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	420.973584	1.036	1.035806
2	218.547207	2.072	1.036182
4	217.743282	1.997	0.499181
8	142.687406	3.067	0.383401
12	132.658072	3.404	0.283687
32	118.249336	3.671	0.114717
64	108.745669	4.268	0.066691
128	100.963180	4.365	0.034105

5 000 000 элементов по возрастанию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	90.794041	1.012	1.012485
2	51.124832	2.017	1.008253
4	47.995604	2.126	0.531462
8	42.345400	2.307	0.288329
12	37.507310	2.729	0.227436
32	42.194875	2.264	0.070758
64	39.114821	2.506	0.039152
128	45.577723	2.062	0.016113

5 000 000 элементов по убыванию

Число потоков (лимит P)	Время исполнения (мс)	Ускорение S	Эффективность E
1	89.182319	1.172	1.171676
2	52.507719	1.840	0.919813
4	50.461045	1.778	0.444401
8	53.800726	1.867	0.233341
12	38.487343	2.484	0.206960
32	37.545527	2.452	0.076619
64	38.557774	2.562	0.040025
128	46.181374	2.144	0.016748

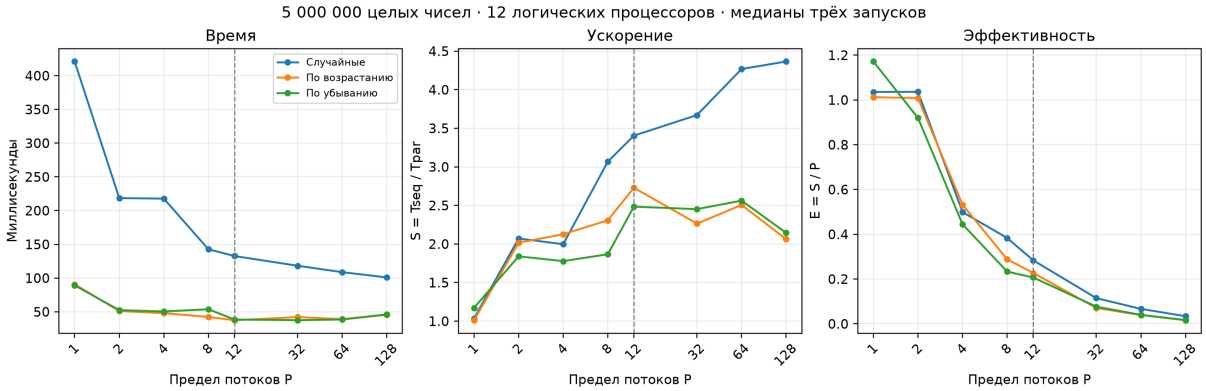
Лимит и фактическое число потоков

Для 5 000 000 случайных элементов ниже приведены последовательные базы, число созданных потоков с главным и наблюдаемый ОС пик. В двух последних столбцах — максимумы среди трёх повторов; это не означает, что максимум наблюдался одновременно с медианным временем. Полные отдельные значения находятся в CSV. Лимит 128 не выдаётся за 128 одновременно работавших потоков.

Лимит P	Tseq, мс	Создано с главным	Наблюдаемый пик
1	436.046812	1	1
2	452.909450	2	2
4	434.773091	4	4
8	437.651470	8	8
12	451.600179	12	12
32	434.088190	32	26
64	464.153020	64	45
128	440.748944	128	68

Графики

Размер 5 000 000, медианы трёх повторов. Ось P логарифмическая по основанию 2; пунктир соответствует 12 логическим процессорам. Графики построены из CSV, отдельные файлы scaling.png, scaling.svg и scaling.pdf пригодны для просмотра и экспорта.



Анализ и выводы

На пяти миллионах случайных чисел при P=12 время составило 132.658072 мс, ускорение 3.404. Наибольшее ускорение среди исследованных пределов для этого входа — 4.365 при P=128, эффективность 0.034105. Значения относятся к конкретной машине и нагрузке, не являются гарантией для других запусков.

Для данных по возрастанию при P=12 измерено 37.507310 мс, при P=128 — 45.577723 мс. Поэтому увеличение предела выше числа процессоров не даёт универсального улучшения; характер данных влияет на пользу от распараллеливания.

Для данных по убыванию при P=12 измерено 38.487343 мс, при P=128 — 46.181374 мс. Поэтому увеличение предела выше числа процессоров не даёт универсального улучшения; характер данных влияет на пользу от распараллеливания.

На 10 000 элементов новые потоки не создаются: весь диапазон меньше 16384. Отношения отдельных коротких времён могут отличаться от единицы из-за прогрева, разрешения часов и фоновой нагрузки, а не параллельной работы. При P=1 оба входа используют последовательный алгоритм, но измеряются в разное время; небольшое S или E выше единицы не доказывает сверхлинейного ускорения.

На больших случайных массивах больше полезной работы на поток и параллельная сортировка половин окупает накладные расходы. Ускорение ограничивают

последовательное финальное слияние, создание и ожидание потоков, общий доступ к памяти, несимметричное распределение бюджета и ограниченные ресурсы CPU. Ожидающие родители входят в лимит, хотя не выполняют сравнения. При большом P растёт стоимость создания и планирования, эффективность S/P обычно падает. Без профилирования нельзя количественно приписать разницу одному фактору.

Три повтора дают предварительную оценку. Последовательная версия всегда выполняется первой, ОС-наблюдение само создаёт небольшую нагрузку, а пик потоков определяется сэмплированием. Для более строгого исследования нужны дополнительные повторы и контроль нагрузки. Условия PDF и дополнительных указаний отражены в реализации, методике и таблицах.